

# Diseño y análisis de algoritmos

Ricardo Wehbe

UADE

15 de agosto de 2026

# Programa

- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

# ¿Dónde estamos?

- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

# Las notaciones $\mathcal{O}$ y $\Theta$

- Notación:  $\mathbb{R}^+$  es el conjunto de los reales *no negativos*.
- Si  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ , entonces  $f(n) \in \mathcal{O}(g(n))$  (el “orden” de  $f$  es  $g$ ) si existen constantes  $c_1$  y  $n_0$  tales que

$$0 \leq f(n) \leq c_1 g(n) \text{ para todo } n \geq n_0$$

- Si  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ , entonces  $f(n) \in \Theta(g(n))$  (el “orden exacto” de  $f$  es  $g$ ) si existen constantes  $c_1$ ,  $c_2$  y  $n_0$  tales que

$$0 \leq c_2 g(n) \leq f(n) \leq c_1 g(n) \text{ para todo } n \geq n_0$$

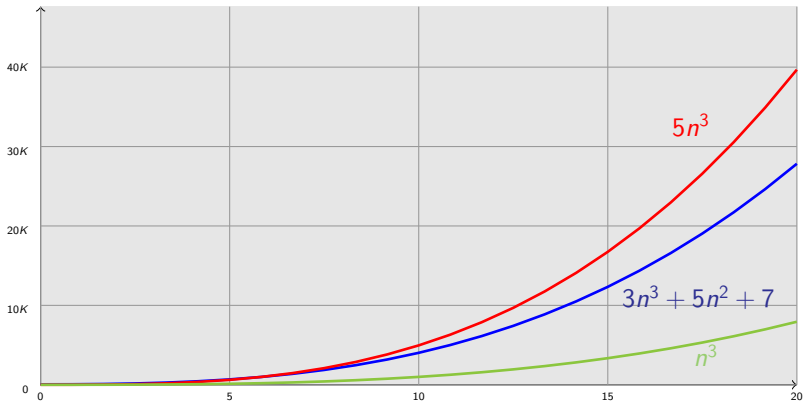
## Un ejemplo. Primera parte

- El objetivo es mostrar que  $f(n) = 3n^3 + 5n^2 + 7 \in \Theta(n^3)$ .
- Para ello, debemos encontrar las constantes  $c_1$ ,  $c_2$  y  $n_0$  tales que

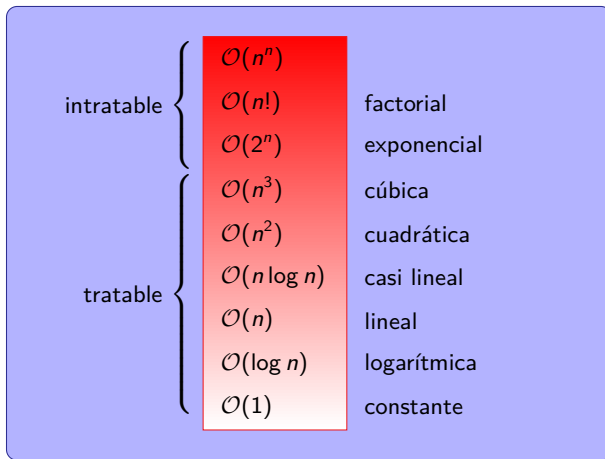
$$c_2 n^3 \leq 3n^3 + 5n^2 + 7 \leq c_1 n^3 \text{ para todo } n \geq n_0$$

- Por ejemplo,  $c_2 = 1$ ,  $c_1 = 5$  y  $n_0 = 3$ .

# Un ejemplo. Segunda parte



# Una jerarquía (parcial) de casos de complejidad



# Resolución de recurrencias: el caso de substracción

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(n^k a^{n/b}) & \text{si } a > 1 \end{cases}$$



# Resolución de recurrencias: el caso de división

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n/b) + p(n) & \text{si } n \geq b \end{cases}$$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log n) & \text{si } a = b^k \\ \Theta(n^{\log_b a}) & \text{si } a > b^k \end{cases}$$

## ¿Dónde estamos?

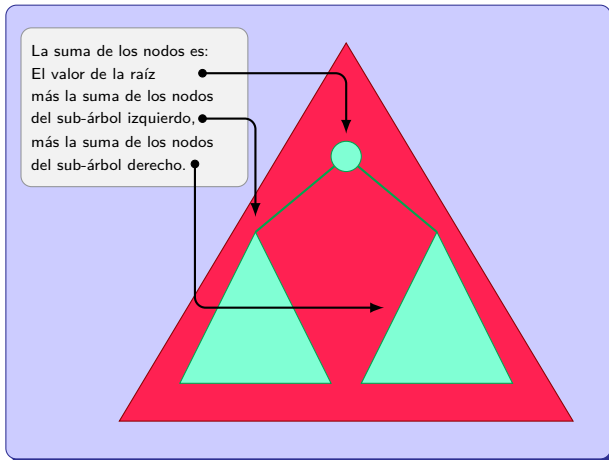
- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

(...) de diviser chacune des difficultés que j'examinerais, en autant de parcelles qu'il se pourrait, et qu'il serait requis pour les mieux résoudre.

((...) dividir los problemas bajo examen en tantas partes como sea posible y necesario para resolverlos mejor.)

René Descartes, *Discours de la méthode*

## Souvenir: ABB

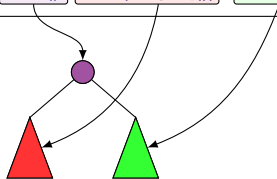


## Intermezzo. El tipo de datos ABB

- Asumimos que tenemos los siguientes métodos de costo constante:
- El método `raiz`, que devuelve el valor de la raíz del árbol.
- Los métodos `hijolq` e `hijoDer`, que devuelven los sub-árboles izquierdo y derecho respectivamente.
- El método `arbolVacio`, que devuelve `true` si el árbol está vacío y `false` en caso contrario.

## Un primer algoritmo *divide & conquer*

```
1.  Algoritmo Suma (ABB T)
2.    if (T.arbolVacio())    //caso base
3.      return 0
4.    else
5.      return T.raiz() + Suma(T.hijoIzq()) + Suma(T.hijoDer())
```



## La técnica *divide & conquer*

- Es una técnica que consiste en dividir un problema “grande” en una serie de problemas “pequeños” de resolución más simple y luego combinar las soluciones de los problemas “pequeños” para obtener una solución del problema “grande.”
- La técnica es recursiva: los problemas “pequeños” son de la misma naturaleza que el problema “grande” y por lo tanto se subdividen a su vez hasta llegar a un problema “mínimo” de resolución trivial (el caso base de recurrencia).
- Se efectúan entonces los siguientes pasos: 1. dividir, 2. conquistar (resolver los problemas mínimos) y 3. combinar las soluciones obtenidas.

## Un algoritmo *divide & conquer* genérico

```
1.  Algoritmo D&C ( $x$ )  
2.      if CasoBase( $x$ ) {  
3.          return SoluciónDirecta ( $x$ )  
4.      } else {  
5.           $\langle x_1, \dots, x_n \rangle = \text{descomponer} (x)$   
6.          for ( $i = 0$ ;  $i < n$ ;  $i++$ ) {  
7.               $y_i = D\&C(x_i)$   
8.          }  
9.          return combinar( $y_1, \dots, y_n$ )  
10.     }
```



## ¿Dónde estamos?

- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

## Búsqueda binaria, de nuevo

- Partimos de un vector  $u[0..n-1]$  de números ordenados ascendentemente y un número  $x$ . El programa debe determinar si  $x$  está en el vector o no.
- Por supuesto se puede resolver por “fuerza bruta”: se recorre secuencialmente el vector y se compara cada elemento  $u[i]$  con  $x$ . Esto tiene complejidad  $\mathcal{O}(n)$ .
- Una solución más eficiente es la *búsqueda binaria*: comparamos  $x$  con el elemento central  $u[k]$ . Si  $x$  es mayor, limitamos nuestra siguiente búsqueda a  $u[k+1..n-1]$  y, si es menor, a  $u[0..k-1]$
- Los casos base son: si el índice inicial queda a la derecha del final, entonces no hemos encontrado el número y debe retornarse *false*; si el vector tiene un solo elemento, es trivial determinar si  $x$  pertenece al vector o no al vector. Y si  $u[k] = x$ , la respuesta es obviamente *true*.

## El algoritmo de búsqueda binaria. Versión simplificada

```
1.  algorithm binrec ( $T$  : array[0.. $n - 1$ ] of int,  $ini$ ,  $fin$ ,  $x$  : int)
2.  if ( $ini > fin$ ) {
3.      return -1;
4.  } else {
5.       $mid = (ini + fin)/2$ 
6.      if ( $x == T[mid]$ )
7.          return  $mid$ ;
8.      else if ( $x < T[mid]$ )
9.          return binrec( $T$ ,  $ini$ ,  $mid - 1$ ,  $x$ );
10.     else
11.         return binrec( $T$ ,  $mid + 1$ ,  $fin$ ,  $x$ );
12. }
```

## Complejidad de la búsqueda binaria

- Observemos que se trata de un algoritmo recursivo con división. Tiene la forma

$$T(n) = \begin{cases} c & \text{si } n = 1 \\ T(n/2) + c & \text{si } n > 1 \end{cases}$$

- Tenemos entonces  $a = 1$ ,  $b = 2$  y  $k = 0$ .
- Por lo tanto,  $a = b^k$  (ya que  $1 = 2^0$ ).
- Entonces  $T(n) \in \Theta(n^k \log n) = \Theta(\log n)$ .

## Cálculo de potencias de la forma $a^{(2^k)}$ con $k > 0$

```
1.  int Algoritmo Potencia (int a, n)
2.      if (n == 2) {                                // Caso base
3.          return a * a;
4.      } else {
5.          return Potencia(a, n/2)*Potencia(a, n/2)
6.      }
```

- Aquí tenemos  $a = 2$ ,  $b = 2$  y  $k = 0$ .
- Por lo tanto, estamos en el caso  $a > b^k$ , ya que  $2 > 2^0$ .
- Entonces  $Potencia(a, n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$ .

## Cálculo de potencia, mejorado

Considere la siguiente versión del programa:

```
1.  int Algoritmo Potencia_2 (int a, n)
2.      if (n == 2) {                                // Base case
3.          return a * a;
4.      } else {
5.          int quad = Potencia_2(a, n/2);
6.          return quad*quad;
7.      }
```

Aquí tenemos  $Potencia\_2(a, n) \in \Theta(\log n)$ . ¿Por qué?

## ¿Dónde estamos?

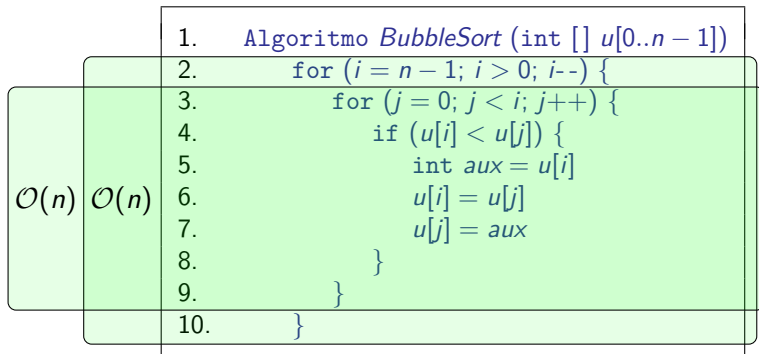
- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

## Algunos métodos de ordenamiento ineficientes

- **Ordenamiento por burbujeo:** consiste en encontrar en la secuencia el mayor elemento y colocarlo en la última posición; se busca luego el mayor de los elementos restantes y se lo coloca en la penúltima posición, y así sucesivamente.
- **Ordenamiento por inserción:** en cada paso  $i$  se coloca el elemento de la posición  $i$  en su posición correspondiente entre las primeras  $i$  posiciones.
- En el primer caso, al final del paso  $i$ , las últimas  $i$  posiciones están ordenadas; en el segundo, al final del paso  $i$ , las primeras  $i$  posiciones están ordenadas.

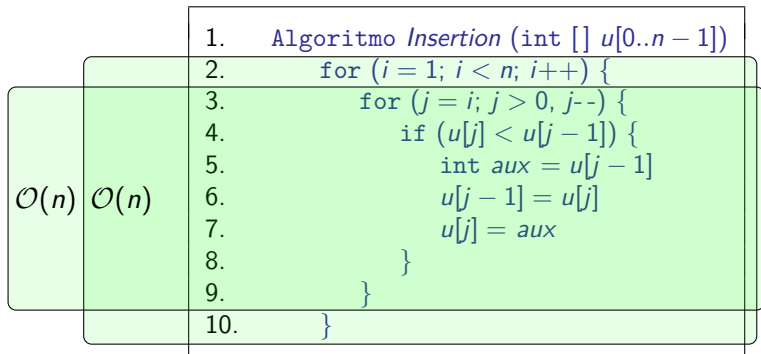


## Ordenamiento por burbujeo



$$\mathcal{O}(n^2)$$

## Ordenamiento por inserción

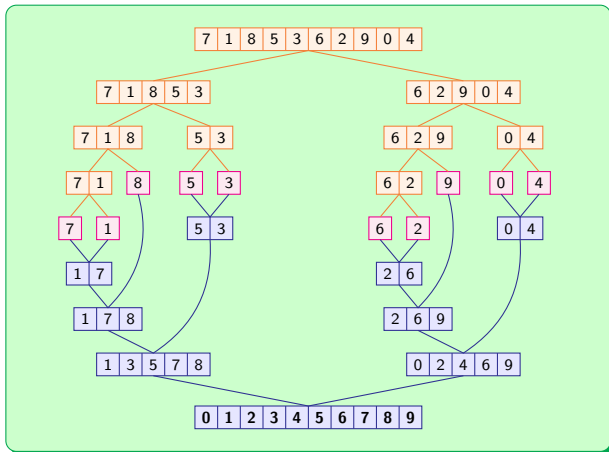


$$\mathcal{O}(n^2)$$

## Métodos de ordenamiento *divide & conquer*

- Veremos dos métodos de ordenamiento *divide & conquer*: *MergeSort* y *QuickSort*.
- *MergeSort* divide la secuencia de entrada en dos mitades, ordena cada una de ellas y luego “mezcla” las mitades ordenadas en una nueva secuencia ordenada. El proceso se aplica recursivamente sobre cada mitad hasta llegar a un caso trivial. Los casos triviales son los vectores de un elemento.
- En el *QuickSort* se elige un elemento (el “pivot”), a partir del cual se divide el vector en dos partes: los elementos mayores al pivot pasan a la derecha del pivot y los menores a su izquierda. Al final de este proceso, el pivot está en su posición definitiva y el proceso continúa recursivamente sobre cada una de las mitades hasta llegar a un caso trivial. El caso trivial es un vector de un elemento.

## Ejemplo de funcionamiento de *MergeSort*



## El algoritmo *MergeSort*

```
1.  Algoritmo MergeSort (int [] u[0..n-1], int ini, fin)
2.      if (ini < fin) {
3.          int mid = (ini + fin)/2
4.          MergeSort (u, ini, mid)
5.          MergeSort (u, mid+1, fin)
6.          Merge (u, ini, fin)
7.      }
```

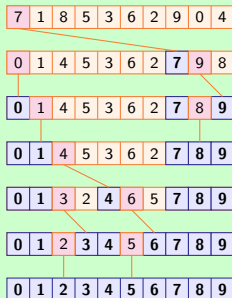
## El algoritmo *Merge*

```
1.  Algoritmo Merge (int [] u[0..n - 1], int ini, fin)
2.      w = int [0..fin - ini];
3.      int mid = (ini + fin)/2;
4.      i = ini;
5.      j = mid + 1;
6.      for (k = 0; k ≤ fin - ini; k++) {
7.          if ((j > fin) || (u[i] ≤ u[j] && i < mid + 1)) {
8.              w[k] = u[i];
9.              i++;
10.         } else {
11.             w[k] = u[j];
12.             j++;
13.         }
14.     }
15.     for (k = 0; k ≤ fin - ini; k++) {
16.         u[ini + k] = w[k];
17.     }
18. }
```

## Complejidad de *MergeSort*

- Para analizar la complejidad de MergeSort, observamos que cada llamada recursiva produce dos llamadas con el argumento dividido por dos. Por lo tanto,  $a = 2$  y  $b = 2$ .
- Para determinar  $k$ , debemos analizar la complejidad de Merge.
- La complejidad de Merge está dada por un ciclo de a los sumo  $n$ . Por lo tanto, tenemos  $\mathcal{O}(n)$  y entonces  $k = 1$ .
- Entonces estamos en el caso  $a = b^k$  y la complejidad es  $\Theta(n \log n)$ .

## Ejemplo de funcionamiento de *QuickSort*





## El algoritmo *QuickSort*

```
1.  Algoritmo QuickSort (int [] u[1..n], int ini, fin)
2.      if (ini < fin) {
3.          integer p = Pivot(u, ini, fin)
4.          QuickSort(u, ini, p - 1)
5.          QuickSort (u, p + 1, fin)
6.      }
```

## El algoritmo *Pivot*

```
1.  Algoritmo Pivot (int []  $u[0..n-1]$ , int  $ini$ ,  $fin$ )
2.       $p = u[ini]$ ;
3.       $i = ini + 1$ ;
4.       $j = fin$ ;
5.      while ( $i \leq fin \ \&\& \ u[i] \leq p$ ) {
6.           $i++$ ;                                 $i$  "junta" valores  $\leq p$ 
7.      }
8.      while ( $u[j] > p$ ) {
9.           $j--$ ;                                 $j$  "junta" valores  $> p$ 
10.     }
11.     while ( $i < j$ ) {
12.         int  $aux = u[j]$ ;
13.          $u[j] = u[i]$ ;
14.          $u[i] = aux$ ;
15.         while ( $i \leq fin \ \&\& \ u[i] \leq p$ ) {
16.              $i++$ ;                                 $i$  "junta" valores  $\leq p$ 
17.         }
18.         while ( $u[j] > p$ ) {
19.              $j--$ ;                                 $j$  "junta" valores  $\geq p$ 
20.         }
21.     }
22.      $u[ini] = u[j]$ ;
23.      $u[j] = p$ ;
24.     return  $j$ ;
25. }
```

## Complejidad de *Pivot*

- La complejidad está dada por el número de iteraciones de los ciclos.
- Los dos primeros ciclos iteran en el peor de los casos  $n$  veces.
- Los ciclos anidados iteran en conjunto en el peor de los casos  $n$  veces.
- Por lo tanto, tenemos una complejidad  $\mathcal{O}(n)$ .

## Complejidad de *QuickSort*

- Consideramos dos casos-límite: cuando el pivot está en la mitad exacta y cuando está en un extremo.
- Caso límite 1: tenemos

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2T(n/2) + p(n) & \text{si } n > 1 \end{cases}$$

Es decir,  $a = 2$ ,  $b = 2$  y  $k = 1$ . Entonces  $T(n) \in \Theta(n \log n)$ .

- Caso límite 2: tenemos

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ T(n-1) + p(n) & \text{si } n > 1 \end{cases}$$

Es decir,  $a = 1$ ,  $b = 1$  y  $k = 1$ . Entonces  $T(n) \in \Theta(n^2)$ .

## Algunas consideraciones sobre el pivot

- La complejidad de *QuickSort* es en el mejor de los casos equivalente a la de MergeSort ( $\mathcal{O}(n \log n)$ ). . .
- . . . y en el peor de los casos comparable a los casos de burbujeo o inserción ( $\mathcal{O}(n^2)$ ).
- La eficiencia de *QuickSort* depende de cuán cerca está el pivot de la media del vector.
- Existen algunas técnicas un poco más elaboradas para seleccionar el pivot.
- Sin embargo, en la práctica a menudo *QuickSort* se desempeña mejor que *MergeSort*. ¿Puede imaginarse por qué?

## Algunas conclusiones

- La idea central de la técnica *divide & conquer* es “deconstruir” un problema en componentes menores que puedan ser más fácilmente resueltos y posteriormente construir una solución combinando las soluciones parciales que se encontraron.
- éste es usualmente un proceso recursivo, que se aplica a los componentes menores, que por lo tanto son también “deconstruidos” en componentes aún menores.
- Por supuesto, necesitamos un caso base como sucede siempre con la recurrencia.
- Los algoritmos de ordenamiento *divide & conquer* están entre los más eficientes.

## ¿Dónde estamos?

- 1 Repaso de la clase anterior
- 2 Introducción a la técnica *divide & conquer*
- 3 Ejemplos de problemas *divide & conquer*
- 4 Métodos de ordenamiento *divide & conquer*
- 5 Ejercicios propuestos

## Ejercicios propuestos 1

- 1 La municipalidad de Fraile Muerto tiene un registro de los  $n$  productores de soja de la región (como todo el mundo sabe, Fraile Muerto está en el corazón de la región sojera de Córdoba.) Los referidos productores están registrados con números secuenciales  $(0, 1, 2, \dots, n-1)$ . Estos datos están almacenados en un vector  $P$ . Pero alguien se percata de que en el vector hay un elemento repetido, es decir que tiene  $n+1$  elementos en lugar de tener  $n$ , aunque el valor del último número es, correctamente,  $n-1$ . Se pide elaborar una estrategia y escribir el programa correspondiente para encontrar de manera eficiente la posición del elemento repetido detallando las diferentes partes que componen la solución y su relación con el esquema general.
- 2 Calcule la raíz cuadrada entera de un número  $n > 0$  utilizando *divide-and-conquer*. Recuerde que la raíz cuadrada entera de un número  $n$  es el máximo valor entero  $u$  tal que  $u^2 \leq n$ . Por ejemplo, la raíz entera de 18 es 4 y la de 9 es 3.



## Ejercicios propuestos 2

- 3 El problema de la moneda falsa. El banco de Fraile Muerto recibió su primera remesa de monedas de cinco pesos. Dentro de un lote de  $n$  monedas se sabe que hay una falsa. La moneda falsa no se distingue de las otras sino por su peso: es un poco más pesada. Se dispone de una balanza de platillos que no da el peso cuantitativo; sólo dice si un grupo de monedas en un platillo pesa más, menos, o lo mismo que otro grupo de monedas en el otro platillo.
- Disponiendo de este *hardware*, más bien modesto, determine el número mínimo de pesadas (en el peor caso) para identificar la moneda falsa.

## Ejercicios propuestos 3

- 4 Picos en un vector. Dado un vector  $u$ , un *elemento pico* es un elemento que no es menor que sus vecinos inmediatos. Encuentre un algoritmo eficiente (mejor que  $\mathcal{O}(n)$ ) que encuentre algún elemento pico en un vector de  $n$  posiciones. Observe que puede haber varios elementos pico; basta encontrar uno de ellos (siempre hay por lo menos uno.) **Ejemplo.** Si la entrada fuera  $[0, 1, 3, 2, 5, 1, 0]$  la respuesta podría ser 3 o 5. Si la entrada fuera  $[7, 2, 3, 4, 5, 6]$  la respuesta podría ser 7 o 6. Si la entrada fuera  $[1, 2, 3]$  la única respuesta sería 3.
- 5 Suponga que  $v$  es un vector de  $n$  dígitos binarios ordenados. Encuentre un algoritmo eficiente (mejor que  $\mathcal{O}(n)$ ) que determine la cantidad de unos en  $v$ . **Ejemplo.** Si  $v = [0, 0, 0, 1, 1, 1, 1, 1]$  (aquí  $n = 8$ ). El programa debería retornar 5, que es la cantidad de unos en  $v$ .